

Software Engineering



Dr. Fatma Eskander
Math.department, Faculty of Science, Mansoura
University

Lecture 2

Software Engineering Processes

Programming and software engineering



Software Engineering processes

The goal of software engineering is not just to write code, but to build reliable, efficient, and maintainable software systematically and predictably. These processes provide the framework to do that.

1. The Waterfall Model: The Traditional Blueprint

This **is a linear, sequential approach**. You must complete one phase fully before moving to the next.

Analogy: Building a House

Requirements Gathering: You meet with an architect and decide you want a 3-bed, 2-bath house with a garage. Everything is documented in a detailed blueprint and a fixed-price contract.

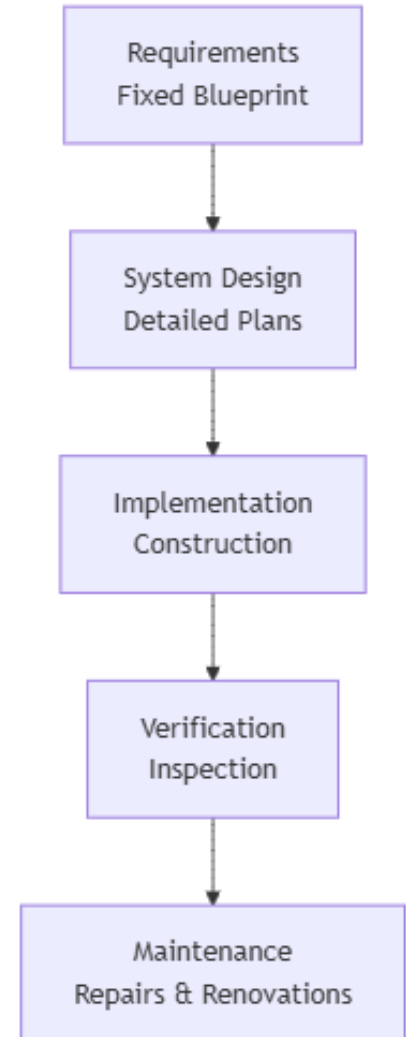
System Design: The architect creates detailed structural, electrical, and plumbing plans. You sign off on them. يضع المهندس المعماري مخططات إنشائية وكهربائية وصحية مفصلة. أنت تُوقع عليها.

Implementation: Construction workers build the house exactly according to the plans (foundation, framing, roofing, etc.).

Verification: The city inspector comes to check if the wiring, plumbing, and structure meet the building codes (the "specification").

Maintenance: You move in. Now, if a pipe bursts or you want to repaint a room, you perform maintenance.

When to use it: When requirements are very well-understood, fixed, and unlikely to change (e.g., safety-critical systems like aircraft controls).



2. The Agile Model: The Iterative & Adaptive Renovation

Agile is an iterative and incremental approach. Instead of building the whole thing at once, you build it in small, functional cycles, getting feedback after each one.

Analogy: Renovating a Kitchen

You don't decide on every single detail upfront. You start with a vision.

لا تُقرر كل تفصيل مُسبقًا. ابدأ برؤية واضحة.

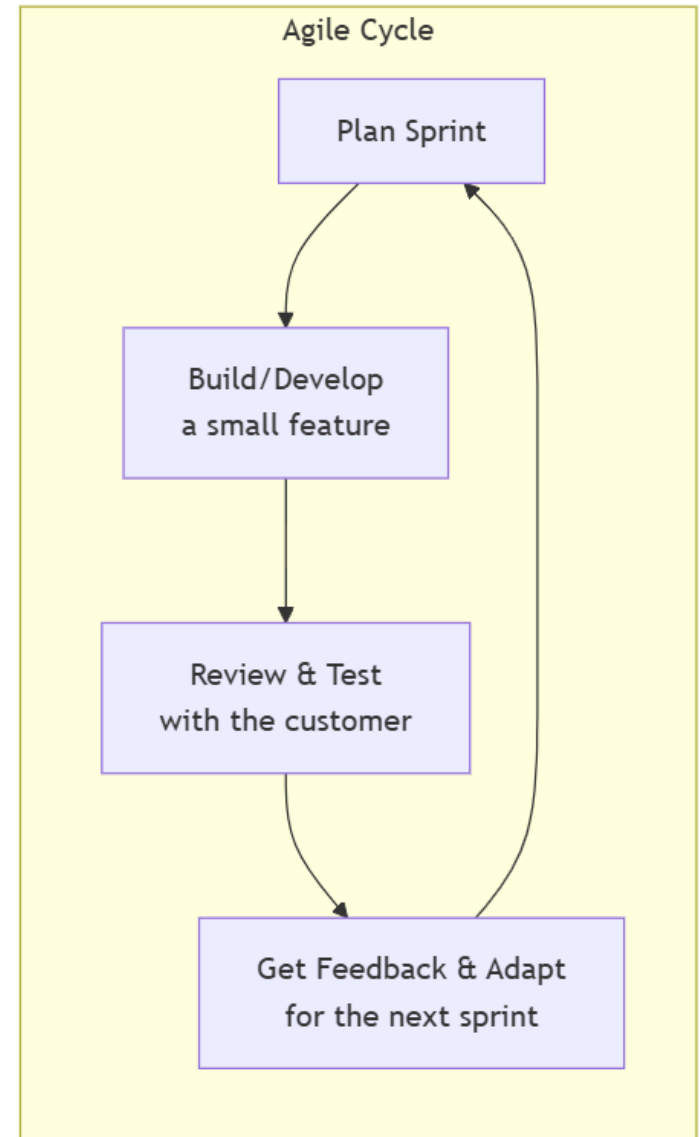
Sprint 1 (2 weeks): Demolish the old kitchen and install the new plumbing and electrical rough-ins. You show the homeowner.

Sprint 2 (2 weeks): Install the cabinets and countertops. The homeowner sees it and says, "Actually, I'd prefer a darker stain on this cabinet."

Sprint 3 (2 weeks): You adjust the cabinet stain and install the flooring. The homeowner loves it.

This process continues for appliances, backsplash, and painting. The final kitchen is a result of continuous collaboration and adaptation.

When to use it: When requirements are expected to change or are not fully known at the start (e.g., most modern web and mobile applications). **Scrum** and **Kanban** are popular frameworks that implement the Agile philosophy.



Scrum framework

Scrum and Kanban are both popular frameworks within the Agile methodology that aim to enhance project management and team collaboration.

Scrum is structured around short, focused work periods called sprints, where teams commit to completing tasks from a prioritized backlog. It emphasizes regular communication through daily stand-up meetings and defined roles such as the Product Owner and Scrum Master.

Kaban framework

Kanban, on the other hand, focuses on visualizing workflow using a board with columns representing different stages of work. It allows for continuous delivery by limiting the number of tasks in progress .

Both frameworks share the core principle of Agile, which emphasizes flexibility, collaboration, and continuous improvement, but they differ in their specific methodologies and approaches to managing work.

3. The DevOps Model: The Fully Integrated Factory

DevOps isn't just a process; it's a cultural shift that combines software Development and IT Operations. It aims to shorten the development lifecycle and provide continuous delivery with high quality.

Analogy: An Automated Car Factory

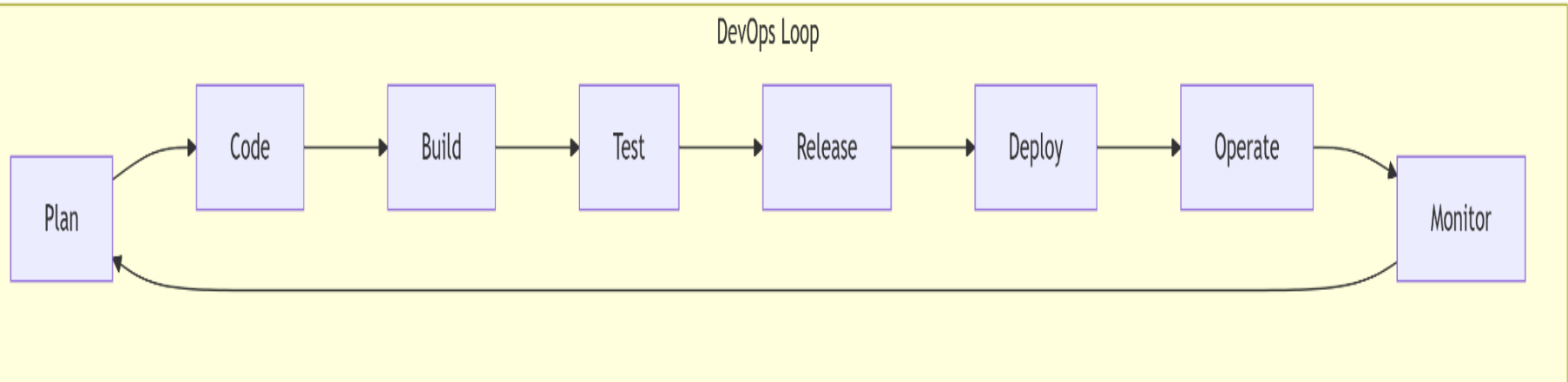
In the old days, one team designed the car (Dev), and another team operated the assembly line (Ops). They often blamed each other for problems.

With DevOps, the designers and the assembly line operators work as one team.

The process is fully automated: when a designer commits a new feature (e.g., a new car door design), it automatically triggers the assembly line robots to build, test, and deploy that new door onto the cars rolling off the line. **Quality is checked at every single step automatically.**

The "Infinity Loop" shows this is a continuous cycle of feedback and improvement.

When to use it: For any service that requires frequent updates, high reliability, and rapid scalability (e.g., e-commerce sites, SaaS products, online banking).



4. The Prototyping Model: The Show Home

This model focuses on building a simplified, working model (a prototype) of the software to gather early feedback from users.

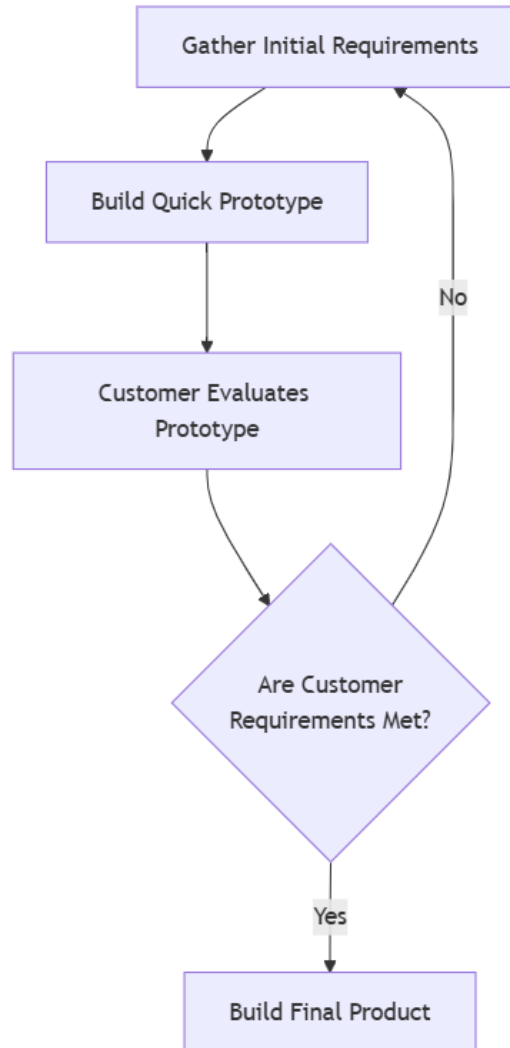
Analogy: Building a Show Home بناء منزل نموذجي

A real estate developer doesn't build the entire neighborhood first. They build one "show home."

Potential buyers walk through it. They might say, "We love the layout, but the bathroom is too small," or "We'd prefer a patio instead of a sunroom."

The developer uses this feedback to refine the architectural plans *before* building all the other houses.

When to use it: When user requirements are vague or when innovating on a new, unproven product concept. عندما تكون متطلبات المستخدم غامضة أو عند الابتكار في مفهوم منتج جديد وغير مثبت.



In practice, modern software teams often use a hybrid approach, like **Agile methodologies within a DevOps culture**, to achieve both speed and reliability.

Summary Table

Process Model	Core Philosophy	Analogy	Best For
Waterfall	Linear, Sequential	Building a House from a Fixed Blueprint	Stable, well-defined requirements.
Agile	Iterative, Adaptive	Renovating a Kitchen in Sprints	Dynamic requirements needing flexibility.
DevOps	Continuous, Automated	An Automated Car Factory	Frequent releases and high scalability.
Prototyping	Feedback-Driven	Building a Show Home	Unclear or innovative product idea

PROCESS ACTIVITIES

Real software processes are inter-leaved sequences of technical, collaborative and managerial activities with the overall goal of specifying, designing, implementing and testing a software system.

The four basic process activities of specification, development (design & implementation), validation and evolution are organized differently in different development processes. In the waterfall model, they are organized in sequence, whereas in incremental development they are inter-leaved.

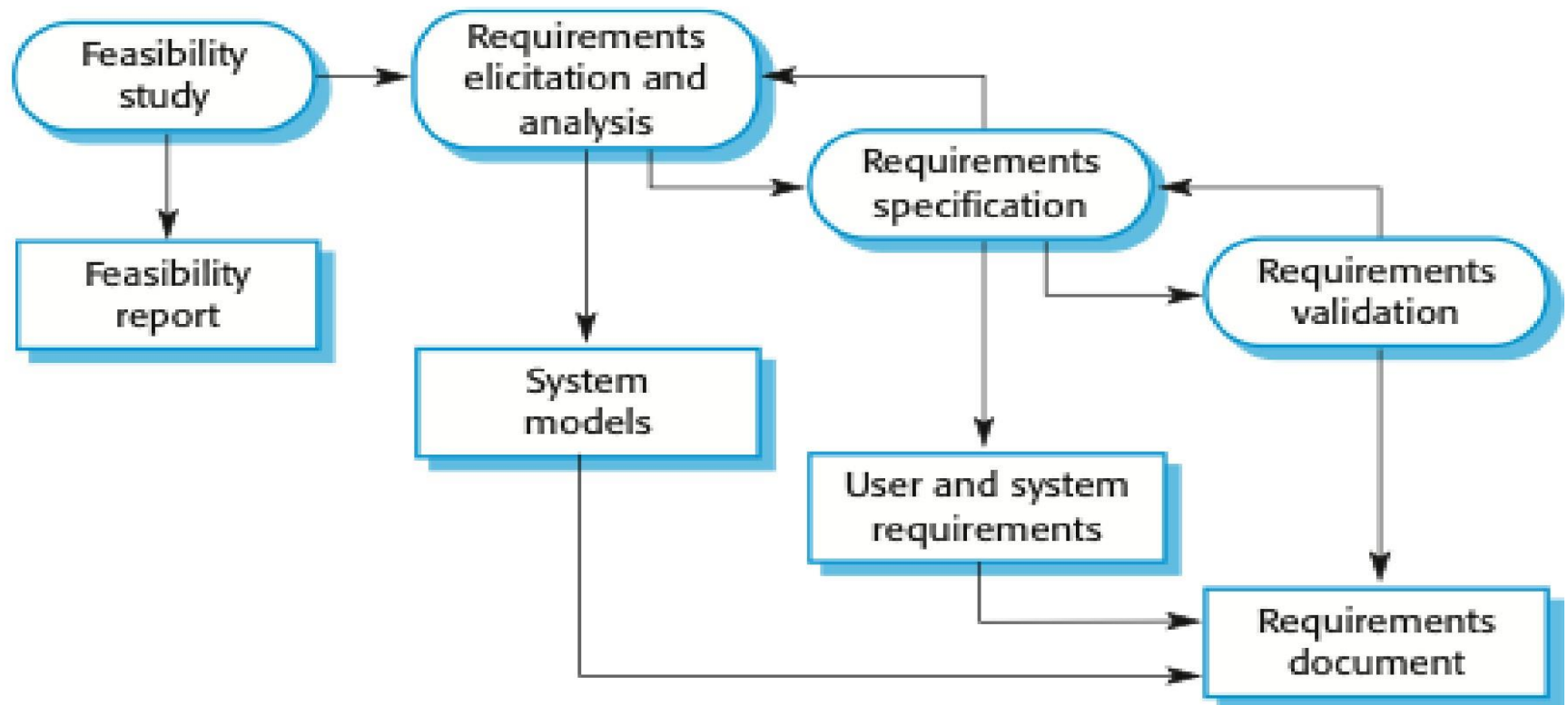
SOFTWARE SPECIFICATION

The process of **establishing what services are required AND the constraints on the system's operation and development.**

Requirements engineering process

- Feasibility study
 - Is it technically and financially feasible to build the system?
- Requirements elicitation and analysis
 - What do the system stakeholders require or expect from the system?
- Requirements specification
 - Defining the requirements in detail
- Requirements validation
 - Checking the validity of the requirements

THE REQUIREMENTS ENGINEERING PROCESS



SOFTWARE DEVELOPMENT (DESIGN AND IMPLEMENTATION)

The process of converting the system specification into an executable system.

Software design

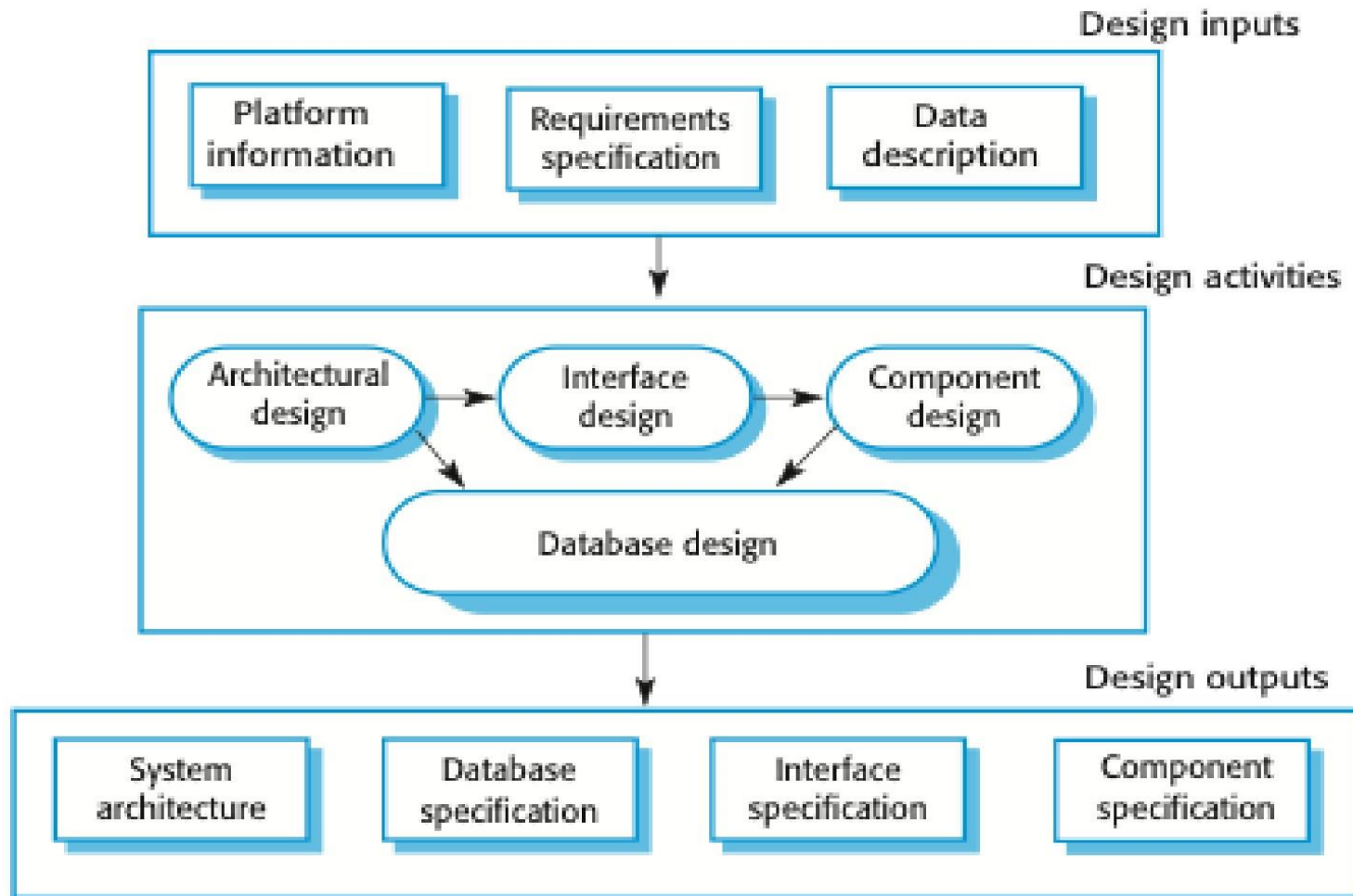
- Design a software structure that realises the specification;

Implementation

- Translate this structure into an executable program;

The activities of design and implementation are closely related and may be inter-leaved.

A GENERAL MODEL OF THE DESIGN PROCESS



DESIGN ACTIVITIES

Architectural design, where you identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships and how they are distributed.

Interface design, where you define the interfaces between system components.

Component design, where you take each system component and design how it will operate.

Database design, where you design the system data structures and how these are to be represented in a database.

SOFTWARE VALIDATION (TESTING)

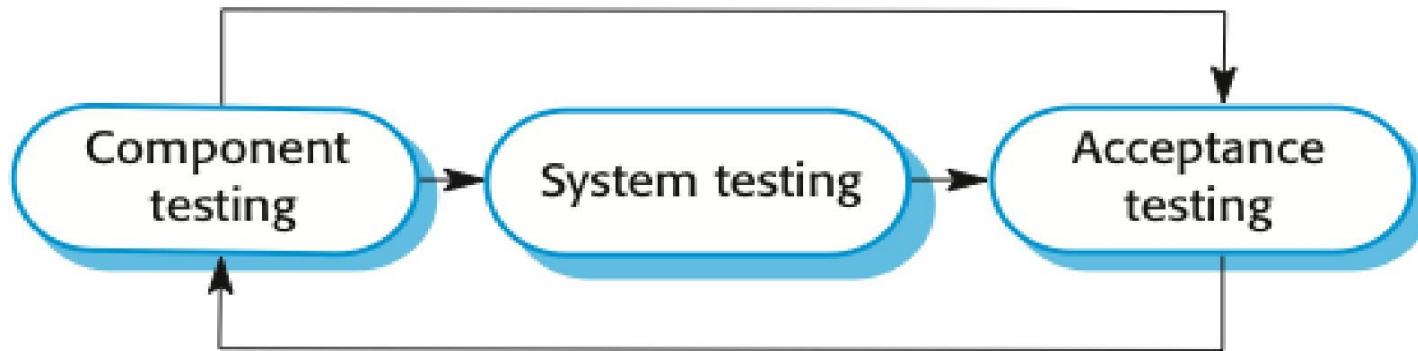
Verification and validation (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer.

Involves checking and review processes and system testing.

System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.

Testing is the most commonly used V & V activity.

STAGES OF TESTING



TESTING STAGES

Development or component testing

- Individual components are tested independently;
- Components may be functions or objects or coherent groupings of these entities.

System testing

- Testing of the system as a whole. Testing of emergent properties is particularly important.

Acceptance testing

- Testing with customer data to check that the system meets the customer's needs.

TESTING PHASES IN A PLAN-DRIVEN SOFTWARE PROCESS

